

ArchAgent

Asistente de IA que guía a equipos de desarrollo **de la idea a una arquitectura de software justificada**

Universidad Pontificia Bolivariana
Vigilada Mineducación

Proyecto aplicado en TIC II

DESARROLLADORES **Daniel Elías Córdoba Howard** • **Laura Sofía Jiménez Moreno** • **Sofía Mejía Rivas** • **Santiago Arias Bernal**

1 Introducción

Definir la arquitectura de un sistema es una de las decisiones más costosas de corregir. En equipos pequeños y académicos suele tomarse sin requerimientos claros, sin comparar alternativas y sin dejar registro de por qué se eligió cada patrón.

ArchAgent es un asistente basado en IA generativa que acompaña al equipo desde la idea del producto hasta una propuesta con **patrones reconocidos, diagramas, trade-offs y un ADR**, dejando al usuario en control de cada decisión.

2 Objetivo

Desarrollar un agente de IA que guíe la definición de arquitecturas de software, desde la **elicitación de requerimientos** hasta una **propuesta justificada** con patrones, diagramas y trade-offs.

Objetivos específicos

- Levantar requerimientos con preguntas progresivas y un resumen aprobable.
- Recomendar patrones con RAG sobre un catálogo curado.
- Generar diagramas Mermaid y tablas de trade-offs.
- Documentar la decisión final en un ADR revisable.
- Permitir que cada usuario conecte su propio LLM.

3 ¿Cómo funciona?

1 Requerimientos
Chat de elicitación guiada y resumen con atributos de calidad.

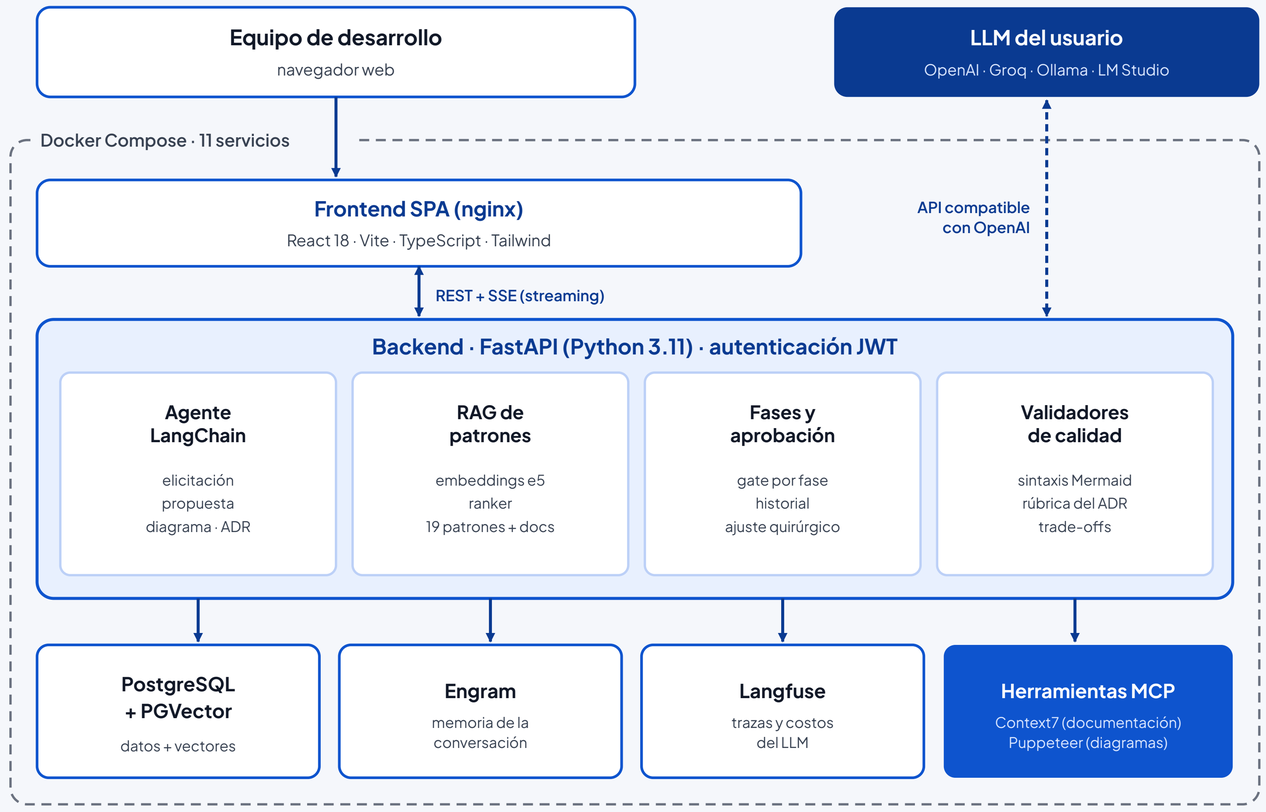
2 Propuesta
Arquitectura con patrones del catálogo citados (RAG).

3 Refinamiento
Diagramas, ajustes puntuales y despliegue en AWS/GCP.

4 Revisión
ADR con contexto, decisión, alternativas y consecuencias.

En cada fase: Aprobar | Modificar | Rechazar

4 Arquitectura



Python 3.11 | FastAPI | LangChain | React 18 | TypeScript | PostgreSQL + PGVector | Mermaid.js | Docker | GitHub Actions

Decisiones de arquitectura y calidad

- **PostgreSQL + PGVector** como única base: datos relacionales y búsqueda semántica juntos.
- **SSE** para mostrar la propuesta mientras se genera, con opción de cancelar.
- **LLM configurable** por usuario, con un wizard que filtra modelos por MMLU.
- **CI en GitHub Actions:** pytest, Vitest y evaluaciones en cada PR; despliegue a Railway.
- **Rúbrica del ADR:** respeta equipo y plazo, cifras con origen, coherencia y alternativas diversas.

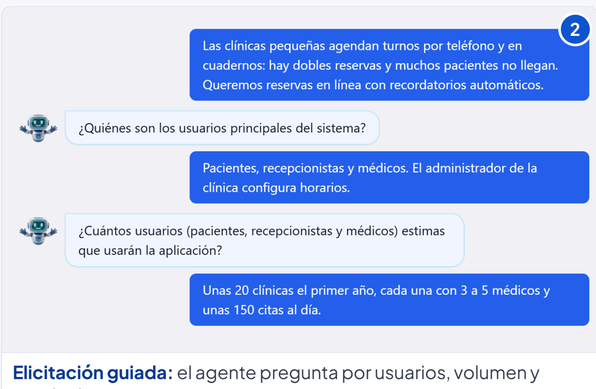
5 Principales resultados

Tras la base de patrones, las capturas siguen el orden de uso con el caso de estudio **TurnoFácil** (citas para clínicas pequeñas) y terminan en el despliegue y el ADR.

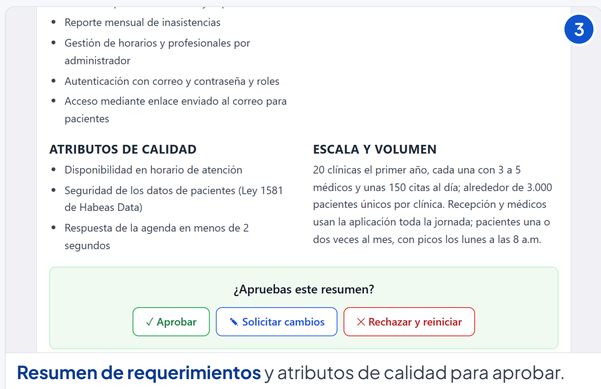
2,4 s propuesta con gpt-oss-20b en Groq (meta: < 5 min)	**19** patrones curados en la base de conocimiento	**−63 %** latencia p50 del RAG: con vs. sin caché, consultas repetidas	**+2.100** pruebas automatizadas en CI (backend + frontend)	**8** casos golden evalúan la propuesta y el ADR



Base de patrones: el catálogo que alimenta el RAG; se consulta en cualquier momento.



Elicitación guiada: el agente pregunta por usuarios, volumen y restricciones.



Resumen de requerimientos y atributos de calidad para aprobar.



Propuesta con patrones y fuentes citadas, y botones Aprobar / Modificar / Rechazar.



Iteración con feedback: el usuario pide un cambio y el agente regenera.

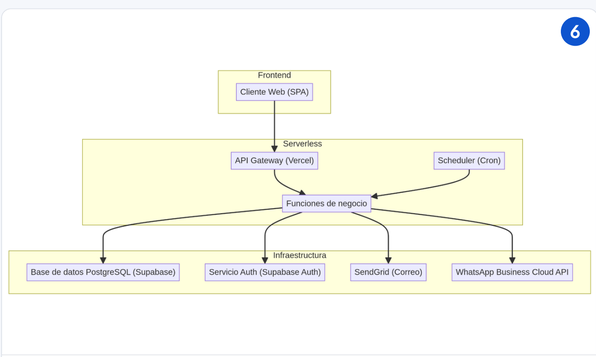
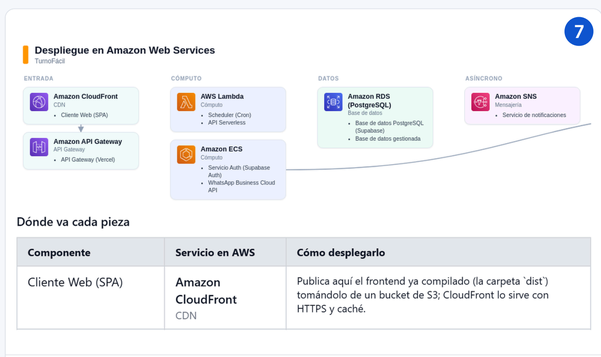


Diagrama de arquitectura generado en Mermaid, con visor y zoom.



Despliegue en la nube: cada componente pasa al servicio equivalente de la nube elegida (AWS o GCP).



ADR: resumen, opciones comparadas, consecuencias y riesgos.

6 Features implementados

Agente y propuesta

- F05 Elicitación guiada
- F06 Pipeline RAG (PGVector)
- F07 Base de patrones
- F08 Propuesta y aprobación
- F09 Diagramas Mermaid
- F10 Tabla de trade-offs
- F19 Optimización del RAG
- F27 Despliegue en AWS/GCP
- F30 Diseño del ADR

Plataforma

- F01 Stack Docker Compose
- F02 Arquitectura técnica
- F03 Caso de ejemplo (seed)
- F04 Mockups de la UI
- F11 Context7 MCP
- F12 Memoria con Engram
- F13 Puppeteer MCP
- F14 Observabilidad (Langfuse)
- F17 Manejo de errores
- F18 Pruebas y CI/CD
- F21 Documentación de usuario

Historias de usuario

- HU1 Registro e inicio de sesión
- HU2 Recordar sesión
- HU3 Lista de proyectos
- HU4 Perfil de usuario
- HU5 Preguntas progresivas
- HU6 Visor de diagramas
- HU8 Justificación con patrones
- HU9 Propuesta en < 5 min
- HU10 Aprobación por fases
- HU11 Ajuste quirúrgico
- HU12 Configuración del LLM
- HU13 Documentos en el RAG

7 Conclusiones

- ArchAgent cubre el ciclo completo: **requerimientos, propuesta, refinamiento y ADR**, con aprobación del usuario en cada fase.
- Apoyar la propuesta en un **catálogo curado** y una rúbrica evita respuestas genéricas o cifras inventadas.
- Con un modelo rápido la propuesta sale en segundos, lo que permite **iterar varias veces** sobre la arquitectura.
- El LLM configurable hace la herramienta **independiente del proveedor**.

8 Pasos futuros

- **Piloto con equipos reales** que lleven su arquitectura hasta la nube.
- **Exportar** el ADR y los diagramas al repositorio del proyecto.
- **Generar artefactos:** OpenAPI, esquema de BD e IaC.
- **Mejorar la búsqueda** de patrones y ampliar el catálogo.
- **Medir de extremo a extremo** en producción (< 3 min).
- **Trabajo colaborativo:** varios usuarios por proyecto.

Pruébalo
Escanea el código y abre ArchAgent desde tu dispositivo.